An LLM Agent for Root-Cause Diagnosis of Network Failures from System and Network Logs

Maria Clara Seixa Scheffel
Research Director
Universidade Federal de Santa Catarina (UFSC)
maria.c.s@grad.ufsc.br

Benjamin Grando Moreira
Academic Advisor
Universidade Federal de Santa Catarina (UFSC)
benjamin.grando@ufsc.br

Claude Opus 4.8 (claude-opus-4-8)
AI Co-Author
Anthropic

Abstract—Root-cause diagnosis of network failures from raw logs is a core NetOps task that remains labour-intensive. We study whether a zero-shot large language model (LLM) agent can classify individual system and network log lines into their underlying root cause, and we compare it against a frozen, published log-parsing baseline (Drain) on an identical held-out split. Evaluation uses a synthetic dataset of 350 log lines across seven root-cause classes, deliberately built with lexical overlap, misleading tokens, and planned-versus-failure ambiguity; it is not real production data. On the 105-line held-out test split, the zero-shot LLM classifier reaches 98.1% accuracy and 0.9809 macro-F1, while the supervised Drain baseline reaches 96.19% accuracy and 0.9624 macro-F1. The margin is small (about 1.9 accuracy points), and Drain – which receives training supervision the LLM does not – wins on two of seven classes. This paper is written by a four-agent LLM pipeline; the multi-agent authorship and its disclosure are described explicitly.

Index Terms—root-cause analysis, log analysis, large language models, NetOps, AIOps, zero-shot classification, network failure diagnosis.

I. Introduction

Network operations (NetOps) teams spend a large fraction of incident time reading logs. When a service degrades, the first evidence is usually a stream of syslog, router console, routing-protocol, and resolver messages, and the operator's task is to infer, from that stream, why the failure occurred. This root-cause diagnosis step is hard because logs are heterogeneous in format, high in volume, and often describe the same underlying fault in very different words: a timeout can surface as an expired routing-protocol timer, packet loss as a rising CRC/FCS counter, and a hardware fault as a bare interface "down" event.

Large language models (LLMs) are attractive for this task because they read heterogeneous text without a fixed schema and can, in principle, reason about context rather than surface keywords. Recent work suggests that LLMs can perform log parsing and anomaly detection with little or no in-domain training [1], [10], and surveys of AIOps and failure management position LLMs as a reshaping force for the field [5], [6]. Whether that generalises to fine-grained root-cause labelling of single log lines, and whether the LLM actually beats a cheap classical parser, is an open and practical question.

This paper is produced by a four-agent pipeline (Researcher, Experimenter, Writer, Reviewer); the Appendix discloses the architecture in full. Our contributions are: (i) a zero-shot LLM agent that classifies each raw log line into one of seven root-cause classes; (ii) a controlled comparison against Drain [3], a frozen published parser used as a supervised baseline on the identical held-out split; and (iii) an honest report – on a synthetic benchmark designed to be hard – showing that the zero-shot LLM narrowly leads a supervised classical baseline, but that the baseline remains competitive and even wins on some classes.

II. Related Work

Classical log analysis. Drain [3] is a widely used online parser that groups raw messages into templates with a fixed-depth parse tree; it underpins many downstream pipelines and is our baseline. DeepLog [4] models log sequences with an LSTM for anomaly detection and diagnosis. The quality of parsing has been shown to materially affect deep anomaly detectors [8], and public benchmark collections such as Loghub [2] standardise evaluation.

LLMs for logs. LogPrompt [1] shows that prompt-engineered, training-free LLMs can parse and detect anomalies interpretably, sometimes beating models trained on thousands of in-domain lines. Comparative studies evaluate several LLMs for parsing [10], and hybrid LLM+ML methods target data-efficient detection on unstable logs [7]. These motivate our zero-shot design.

LLM agents for incident management. Surveys map the AIOps and microservice fault-localisation landscape [5], [6]. A holistic view of AI-driven network incident management argues for modular LLM architectures [9], and

end-to-end frameworks such as LogSage [11] detect and remediate failures from CI/CD logs with industrial validation. Our work is narrower: single-line root-cause labelling, benchmarked against a frozen classical parser.

III. Method

A. Task and classes

We frame root-cause diagnosis as single-line classification: given one raw log line, predict exactly one of seven labels: timeout, packet_loss, config_drift, auth_failure, dns_failure, hardware_fault, and normal.

B. Synthetic dataset

All data are synthetic. The dataset contains 350 log lines, 50 per class, generated from a fixed seed (20260821). For each class, lines are sampled from 10–14 hand-written templates spanning heterogeneous real formats – syslog, Cisco IOS/Junos console, BGP/OSPF/BFD, systemd/journald, BIND/unbound/dnsmasq/sssd resolvers, and kernel/EDAC/mcelog/SMART. Volatile fields (timestamps, hostnames, interfaces, IPv4 addresses, PIDs, ports, counters, percentages, latencies) are randomised per entry.

The generator deliberately injects difficulty. About half the templates per class are hard: they avoid the obvious class token and instead express the root cause the way real logs do – a timeout as an OSPF dead-timer or BGP hold-time expiry, packet loss as CRC/FCS errors or a high retransmission ratio, an auth failure as a rejected key or certificate, a DNS failure phrased as "timed out" or "no PTR record", and a hardware fault surfaced as a generic link "down" or controller reset. The normal class contains routine chatter that mimics failure vocabulary – successful retries, recoveries, planned administrative shutdowns, approved config changes, established adjacencies – so that naive single-token grepping is actively wrong on many lines and correct classification often requires context. We emphasise that these results say nothing directly about real production logs.

We use a stratified split with a fixed seed: 70% train (245 lines) and 30% test (105 lines, 15 per class).

C. Zero-shot LLM classifier

The LLM agent classifies each raw line with a single claude-haiku-4-5 API call. A fixed system prompt defines all seven labels with short operational descriptions, warns explicitly about misleading tokens and planned-versus-failure ambiguity, and requires the model to reply with only the single label. The user message is the raw log line. No labelled examples are provided: the classifier is zero-shot and sees no training data.

D. Baseline: Drain (supervised)

Our baseline is Drain [3], the published fixed-depth-tree log parser, run as a frozen implementation with published default hyper-parameters (depth 4, similarity threshold 0.4, max children 100; not tuned on this dataset). Two facts must be stated plainly:

TABLE I
Overall results on the held-out test split ($n = 105$).

Metric	LLM zero-shot	Drain (supervised)
Accuracy	0.9810	0.9619
Macro precision	0.9832	0.9699
Macro recall	0.9810	0.9619
Macro F1	0.9809	0.9624
Parse failures	0	0
Unmatched templates	—	4 (3.81%)

TABLE II
Per-class F1 on the test split. Support is 15 per class.

Class	LLM zero-shot	Drain (supervised)
timeout	1.0000	1.0000
packet_loss	0.9286	1.0000
config_drift	1.0000	0.8889
auth_failure	1.0000	0.8824
dns_failure	1.0000	1.0000
hardware_fault	0.9375	1.0000
normal	1.0000	0.9655

- Drain is a parser, not a classifier. It groups lines into templates; we map each template to a class by majority vote over the labelled training split. On our data it learned 78 templates.
- The baseline therefore receives supervision, while the LLM classifier is zero-shot. This asymmetry favours the baseline. We recall it in the Discussion rather than burying it.

We chose Drain deliberately. An earlier iteration of this experiment used keyword rules authored by the same agent that generated the logs; it scored perfectly because it knew its own templates – a conflict of interest. Drain removes that confound: its algorithm and hyper-parameters are fixed by prior published work and contain no notion of the seven failure classes. Templates that appear only at test time are unmatched; we assign them a fixed fallback label (auth_failure).

IV. Results

Both systems are evaluated on the identical 105-line held-out test split. Table I reports accuracy and macro precision, recall, and F1. The zero-shot LLM reaches 98.1% accuracy and 0.9809 macro-F1; the supervised Drain baseline reaches 96.19% accuracy and 0.9624 macro-F1. The LLM leads by 0.0191 in accuracy and 0.0185 in macro-F1. The LLM produced no parse failures across all 350 generation-and-test calls; Drain left 4 of 105 test lines unmatched (3.81%), routed to the fallback label.

Table II gives per-class F1. The LLM's only errors were two packet_loss lines misread as hardware_fault (recall 0.8667 for packet_loss; precision 0.8824 for hardware_fault). Drain's errors were three config_drift lines mapped to auth_failure and one normal line likewise,

TABLE III
Split sensitivity as the baseline's supervision changes. LLM is zero-shot throughout.

Train	n_{test}	LLM F1	Drain F1	Drain unmatched
0.4	210	0.9809	0.8987	0.1524
0.5	175	0.9770	0.9302	0.1086
0.6	140	0.9711	0.9718	0.0286
0.7	105	0.9809	0.9624	0.0381
0.8	70	0.9857	1.0000	0.0000

driving auth_failure precision down to 0.7895. Drain strictly wins on packet_loss and hardware_fault; the LLM strictly wins on config_drift, auth_failure, and normal. The zero-shot classification of all 350 lines took 35.4 s of wall-clock time over 350 calls (116,637 input and 1,902 output tokens), with no retried API errors.

Table III shows how the comparison shifts with the training fraction. With little supervision (40% train) Drain's unmatched rate rises to 15.24% and its macro-F1 falls to 0.8987, while the zero-shot LLM is unaffected (0.9809). As supervision grows, Drain catches up and overtakes: at 80% train it reaches a perfect 1.0000 on the smaller 70-line test split, above the LLM's 0.9857.

V. Discussion

Where the LLM does better. The zero-shot LLM leads overall and is markedly more robust when supervision is scarce (Table III). Because it requires no template-to-class map, it has no unmatched-template failure mode: a test line in a format never seen at training time is still classified, whereas Drain must fall back blindly. The LLM's wins on config_drift, auth_failure, and normal are exactly the classes where surface tokens mislead – approved changes, rejected credentials, and planned operations that resemble failures – suggesting the model uses context rather than keywords.

Where the published baseline is already sufficient or better. Drain is genuinely competitive. On the 70/30 split its macro-F1 is 0.9624, only 0.0185 below the LLM, and it wins outright on packet_loss and hardware_fault. With ample supervision it matches or beats the LLM (Table III, 60% and 80% train). This is a frozen, decade-old, essentially free parser with no notion of the failure classes, and it is supervised where the LLM is not – an asymmetry that favours it, as stated in the Method. We do not rescue the LLM with speculation: on this synthetic task the classical parser is close.

Actual added value. The LLM's concrete advantage here is data efficiency and robustness, not a large accuracy gap. When labelled data are plentiful and log formats are stable, a supervised parser is nearly as good and far cheaper. The LLM's value appears when supervision is scarce or formats drift – precisely the low-training- fraction regime where its lead is widest and Drain's unmatched rate climbs.

VI. Limitations and Threats to Validity

The dataset is synthetic and was authored by the same agent pipeline that evaluates it; this is an internal-validity threat and the numbers should not be read as evidence about real production logs. We use a single generation seed, a single held-out test split of only 105 lines (15 per class), and a single LLM (claude-haiku-4-5), so variance is unquantified. The two systems are not on equal footing: the baseline is supervised while the LLM is zero-shot, which favours the baseline yet still leaves the LLM ahead overall. Finally, we compare against a classical parser but not against a trained ML baseline such as a deep sequence model (e.g., an LSTM classifier); such a comparator could close or reverse the small gap we observe.

VII. Conclusion

On a deliberately hard, synthetic seven-class benchmark, a zero-shot LLM agent classified raw log lines into their root cause at 98.1% accuracy and 0.9809 macro-F1, narrowly ahead of a frozen, supervised Drain baseline at 96.19% and 0.9624. The margin is small, the baseline wins on two classes, and it is competitive despite – or because of – the supervision it receives and the LLM does not. The LLM's clearest benefit is robustness under scarce supervision and format drift. Validating these findings on real network logs, with a trained ML baseline and multiple seeds and models, is the essential next step.

Appendix

Disclosure Statement

Multi-agent architecture. This paper was produced by a four-node LangGraph pipeline: a Researcher that gathers and verifies sources, an Experimenter that runs the measurements, a Writer that drafts the paper, and a Reviewer that checks the draft. Explicit state (verified sources, results JSON, draft) is passed between nodes, with a review-feedback loop back to the Writer for correction.

Models and frameworks. Orchestration uses LangGraph and LangChain over the Anthropic API. The Researcher, Writer, and Reviewer agents run on claude-opus-4-8. The high-volume per-log classification calls use claude-haiku-4-5, chosen specifically to bound API cost for that repeated task. The Researcher used server-side web search and web-fetch tools.

Baseline classifier. A non-LLM published comparator, Drain (He et al., ICWS 2017 [3]), was run on the identical held-out test split. It is a frozen implementation using published default hyper-parameters and contains no class-specific rule; templates are mapped to classes by majority vote over the labelled training split. It replaced an earlier agent-authored keyword baseline, which was discarded as confounded because it was written by the same agent that generated the logs. Drain receives training supervision that the zero-shot LLM does not.

Author-resolution retry step. When a retrieved source's authors were not immediately determinable, the Researcher performed at least one additional targeted search or a direct fetch of the publisher/arXiv page before any author could be recorded as unknown. Every cited source here was author-confirmed via such follow-up.

Constraints applied. No invented citations and no invented results were permitted: every citation is a Researcher-verified source and every number comes from the Experimenter's output. Data are synthetic and labelled as such throughout. The Reviewer agent verified citations and numbers against the Researcher's and Experimenter's outputs and returned the draft for correction when it found mismatches.

Division of labour. The human authors (M. C. S. Scheffel and B. G. Moreira) defined the research question, supervised the pipeline, and take final responsibility for the work. The AI co-author (Claude Opus 4.8) executed the Researcher, Writer, and Reviewer roles and drafted this manuscript; the Haiku model performed the per-log classification calls.

References

[1] Y. Liu, S. Tao, W. Meng, J. Wang, W. Ma, Y. Zhao, Y. Chen, H. Yang, Y. Jiang, and X. Chen, "LogPrompt: Prompt Engineering Towards Zero-Shot and Interpretable Log Analysis," arXiv preprint (also ICSE 2024 Companion Proceedings), 2023. [Online]. Available: https://arxiv.org/abs/2308.07610

[2] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics," arXiv preprint, 2020. [Online]. Available: https://arxiv.org/abs/2008.06448

[3] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An Online Log Parsing Approach with Fixed Depth Tree," IEEE International Conference on Web Services (ICWS), 2017. [Online]. Available: https://dblp.dagstuhl.de/rec/conf/icws/HeZZL17.html

[4] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," ACM SIGSAC Conference on Computer and Communications Security (CCS), 2017. [Online]. Available: https://dblp.org/rec/conf/ccs/Du0ZS17.html

[5] L. Zhang, T. Jia, M. Jia, Y. Wu, A. Liu, Y. Yang, Z. Wu, X. Hu, P. S. Yu, and Y. Li, "A Survey of AIOps for Failure Management in the Era of Large Language Models," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2406.11213

[6] S. Zhang, S. Xia, W. Fan, B. Shi, X. Xiong, Z. Zhong, M. Ma, Y. Sun, and D. Pei, "Failure Diagnosis in Microservice Systems: A Comprehensive Survey and Analysis," arXiv preprint (also ACM Transactions on Software Engineering and Methodology), 2024. [Online]. Available: https://arxiv.org/abs/2407.01710

[7] F. Hadadi, Q. Xu, D. Bianculli, and L. Briand, "LLM meets ML: Data-efficient Anomaly Detection on Unseen Unstable Logs," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2406.07467

[8] Z. A. Khan, D. Shin, D. Bianculli, and L. Briand, "Impact of Log Parsing on Deep Learning-Based Anomaly Detection," arXiv preprint, 2023. [Online]. Available: https://arxiv.org/abs/2305.15897

[9] P. Hamadanian, B. Arzani, S. Fouladi, S. K. R. Kakarla, R. Fonseca, D. Billor, A. Cheema, E. Nkposong, and R. Chandra, "A Holistic View of AI-driven Network Incident Management," ACM Workshop on Hot Topics in Networks (HotNets), 2023. [Online]. Available: https://conferences.sigcomm.org/hotnets/2023/papers/hotnets23_hamadanian.pdf

[10] M. Astekin, M. Hort, and L. Moonen, "A Comparative Study on Large Language Models for Log Parsing," ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), 2024. [Online]. Available: https://arxiv.org/abs/2409.02474

[11] W. Xu, J. Luo, T. Huang, K. Sui, J. Geng, Q. Ma, I. Akasaka, X. Shi, J. Tang, and P. Cai, "LogSage: An LLM-Based Framework for CI/CD Failure Detection and Remediation with Industrial Validation," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2506.03691